

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

" ____ " _____ 20 ____ г.

Заведующий кафедрой

_____М.А. Митрохин

ОТЧЕТ ПО УЧЕБНОЙ (ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКЕ
(2025/2026 учебный год)

_____Майер Владислав Сергеевич

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 2 _____ семестр _____ 4 _____

Период прохождения практики с 19.06.2026 по 16.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики к.т.н., доцент, Карамышева Н.С.

(должность, ученая степень, ученое звание)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

" ____ " _____ 20 ____ г.

Заведующий кафедрой

_____ М.А. Митрохин

**ИНДИВИДУАЛЬНЫЙ ПЛАН ПРОХОЖДЕНИЯ УЧЕБНОЙ
(ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКИ**

(2025/2026 учебный год)

_____ Майер Владислав Сергеевич

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 2 _____ семестр _____ 4 _____

Период прохождения практики с 19.06.2026 по 16.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А. _____

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики к.т.н., доцент, Карамышева Н.С.

(должность, ученая степень, ученое звание)

№ п/п	Планируемая форма работы во время практики	Количество часов	Календарные сроки проведения работы	Подпись руководителя практики от вуза
1	Выбор темы и разработка индивидуального плана проведения работ	26	19.06.26 – 24.06.26	
2	Подбор и изучение материала по теме работы	26	24.06.26 – 26.06.26	
3	Установка виртуальной машины	26	26.06.26 – 28.06.26	
4	Установка операционной системы	30	01.07.26 – 04.07.26	
5	Разработка программы на языке Си	30	04.07.26 – 09.07.26	
6	Тестирование и отладка	38	09.07.26 – 12.07.26	
7	Оформление отчёта	40	12.07.26 – 16.07.26	
	Общий объём часов	216		

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЁТ
О ПРОХОЖДЕНИИ УЧЕБНОЙ (ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Майер Владислав Сергеевич

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 2 семестр 4

Период прохождения практики с 19.06.2026 по 16.07.2026

Кафедра «Вычислительная техника»

Майер В.С. выполнял практическое задание «Сортировка пузырьком (bubble sort)». На первоначальном этапе была изучена теоретическая часть работы: рассмотрены основные алгоритмы сортировки массивов, в частности метод пузырька, его принцип работы, преимущества и недостатки. Также была изучена интегрированная среда разработки Qt Creator, её интерфейс и основные инструменты, включая редактор кода с подсветкой синтаксиса и автодополнением, встроенный компилятор Clang, системы сборки CMake и QMake, а также инструменты отладки GDB. После этого была разработана программа на языке C, реализующая сортировку массива методом пузырька. Программа включает в себя ввод количества элементов и самих элементов массива с клавиатуры, сортировку массива по возрастанию, вывод исходного и отсортированного массивов на экран, а также запись результата в текстовый файл. В процессе написания программы была произведена её компиляция, отладка и тестирование в среде Qt Creator. После завершения разработки программы был оформлен отчет о проделанной работе

Бакалавр Майер В.С. " ____ " ____ 2026 г.

Руководитель Карамышева Н.С. " ____ " ____ 2026 г.
практики

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЗЫВ

О ПРОХОЖДЕНИИ УЧЕБНОЙ (ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Майер Владислав Сергеевич

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 2 семестр 4

Период прохождения практики с 19.06.2026 по 16.07.2026

Кафедра «Вычислительная техника»

В процессе выполнения практики Майер В.С. решал следующие задачи: изучение алгоритма сортировки пузырьком и его реализация на языке программирования С; разработка программы для сортировки массива целых чисел с вводом данных с клавиатуры; вывод исходного и отсортированного массивов на экран; запись полученных результатов в текстовый файл; изучение интегрированной среды разработки Qt Creator, включая её основные инструменты и функции; компиляция, отладка и тестирование разработанной программы в среде Qt Creator; оформление отчета о проделанной работе с описанием всех этапов разработки, листингом программы и результатами её выполнения.

За период выполнения практики были освоены основы работы в интегрированной среде разработки Qt Creator, навыки написания программ на языке С с использованием массивов, циклов и условных операторов, практические навыки компиляции и отладки программ, методы работы с файловым вводом-выводом. Во время выполнения работы Майер В. С. показал себя ответственным, добросовестным учеником, знающим свой предмет, имеющим представление о современном состоянии науки, владеющим современными общенаучными знаниями по информатике и вычислительной технике.

За выполнение работы Майер В. С. заслуживает оценки « ».

Содержание

Введение	2
1. Установка ОС Linux на виртуальную машину.....	3
1.1 Установка виртуальной машины VirtualBox.....	3
1.2 Создание виртуальной машины	6
1.3 Установка ОС Ubuntu на виртуальную машину	10
1.4 ОС Ubuntu	13
2. Интегрированная среда разработки Qt Creator	15
2.1 Описание Qt Creator	15
2.2 Описание установки.....	16
3. Разработка программы на языке C в среде Qt Creator.....	18
3.1 Формулировка задачи	18
3.2 Создание проекта в Qt Creator	18
3.3 Описание программы.....	21
3.4 Компиляция и отладка программы в среде Qt Creator	22
Заключение	25
Список использованных источников и литературы	26
Приложение А	27
Приложение Б	29

Введение

Linux, или GNU/Linux — семейство Unix-подобных операционных систем на базе ядра Linux, включающих тот или иной набор утилит и программ проекта GNU. В Linux есть много ответвлений, дистрибутивов таких, как Ubuntu, Linux Mint, Fedora, CentOS, Debian и другие. Большинство из них совершенно бесплатны и распространяются с открытым исходным кодом, что делает их более удобными для разработчиков программного обеспечения. Основным преимуществом ОС Linux является безопасность, отсутствие вирусов и угрозы со стороны вредоносных программ. Популярные дистрибутивы Linux очень просты в установке и не требовательны к ресурсам. Linux идеален для программистов и веб-разработчиков, поскольку имеет встроенные компиляторы, удобную командную строку, через которую можно даже устанавливать популярные приложения. Linux прекрасно работает как на обычных домашних компьютерах, так и на рабочих компьютерах и серверах.

Виртуальная машина (ВМ или VM) — это виртуальный компьютер, который использует выделенные ресурсы реального компьютера (процессор, диск, адаптер). Эти ресурсы хранятся в облаке и позволяют ВМ работать автономно. ВМ может работать в отдельном окне как программа или запускаться через панель управления. Виртуальная машина может использоваться для развертывания двух и более операционных систем на одном физическом устройстве. Таким образом, установив на виртуальную машину операционную систему Linux, можно работать в ней, не перезагружая компьютер с основной ОС Windows. К тому же ВМ дает возможность одновременной работы в нескольких операционных системах, а также взаимодействие между ними. Еще одним достоинством виртуальной машины является то, что работа в ней никак не навредит реальному компьютеру.

Задачей данной практической работы стала установка операционной системы Linux на виртуальную машину и разработка программы на языке C в установленной в данной ОС интегрированной среде разработки.

1. Установка ОС Linux на виртуальную машину

1.1 Установка виртуальной машины VirtualBox

Virtualbox — кроссплатформенное программное обеспечение, обладающее понятным и дружелюбным интерфейсом. Для установки виртуальной машины скачали с официального сайта установочный файл и запустили его (Рисунок 1).



Рисунок 1 — Запуск установочного файла VirtualBox

Затем выбрали компоненты для установки и расположения (Рисунок 2).

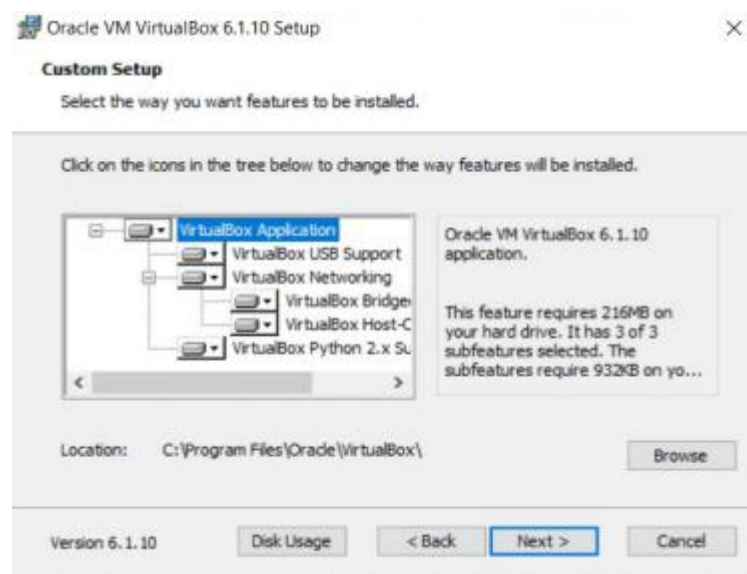


Рисунок 2 — Выбор компонентов для установки и расположения

Далее отметили «Создать ярлык на рабочем столе» (Рисунок 3).

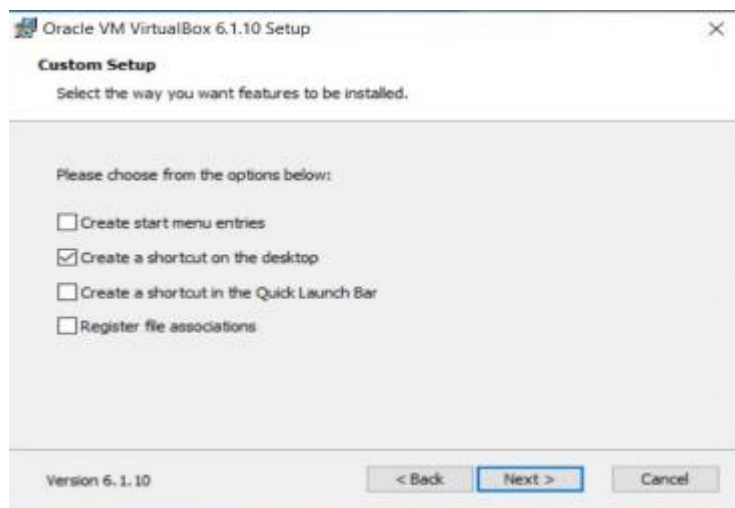


Рисунок 3 — Создание ярлыка на рабочем столе

Потом согласились на отключение Интернет-соединения на время установки VirtualBox (Рисунок 4).



Рисунок 4 — Временное отключение Интернет-соединения

Затем установили программное обеспечение для виртуальной машины (Рисунок 5).

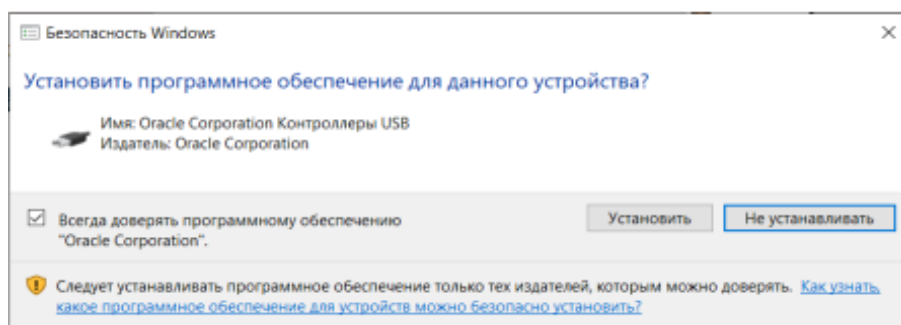


Рисунок 5 — Установка программного обеспечения для ВМ

Далее нажали кнопку «Завершить» для выхода из Мастера установки (Рисунок 6).



Рисунок 6 — Выход из Мастера установки

Потом установили VirtualBox (Рисунок 7).

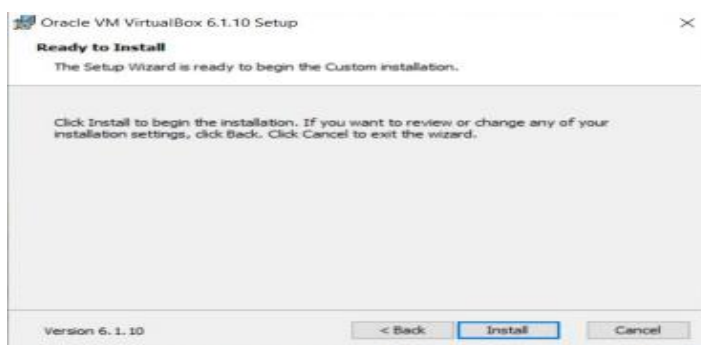


Рисунок 7 — Установка VirtualBox

Установка VirtualBox была успешно завершена, для проверки запустили ВМ (Рисунок 8).

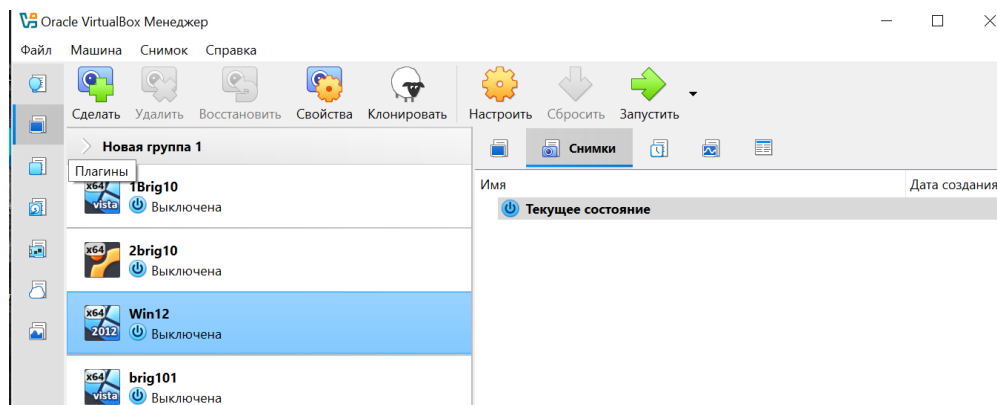


Рисунок 8 — Запуск VirtualBox

1.2 Создание виртуальной машины

Для создания виртуальной машины запустили VirtualBox и нажали кнопку «Создать» (Рисунок 9).

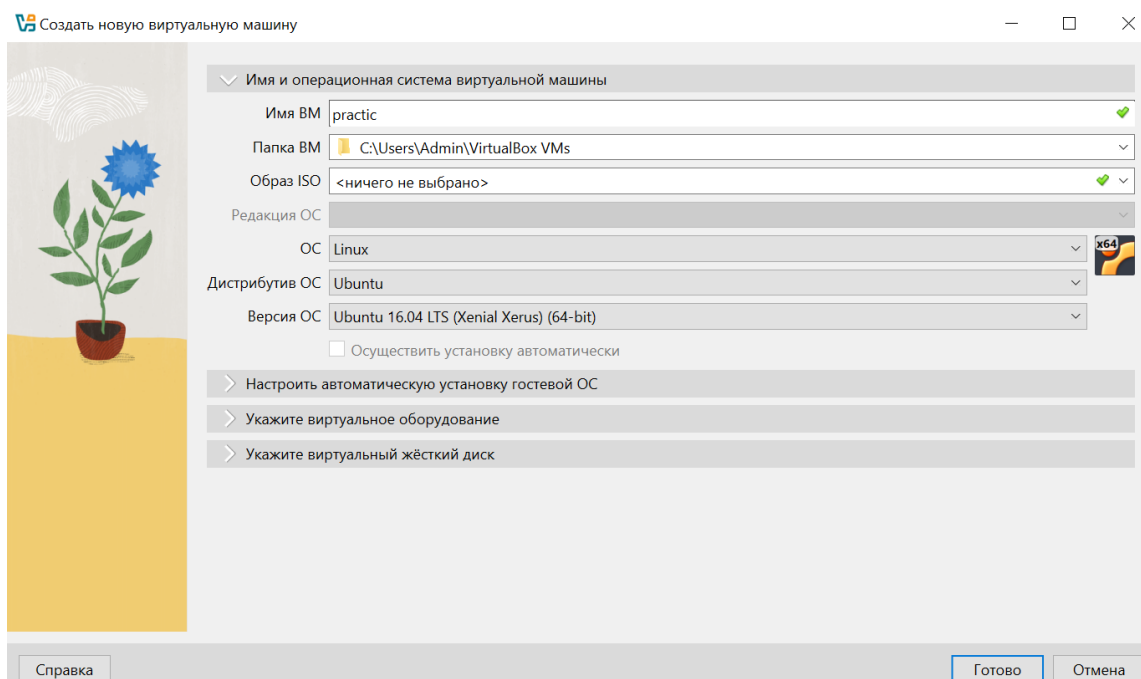


Рисунок 9 — Создание виртуальной машины

Указали имя, местоположение, тип и версию операционной системы новой виртуальной машины. Далее указали объем оперативной памяти виртуальной машины (Рисунок 10).

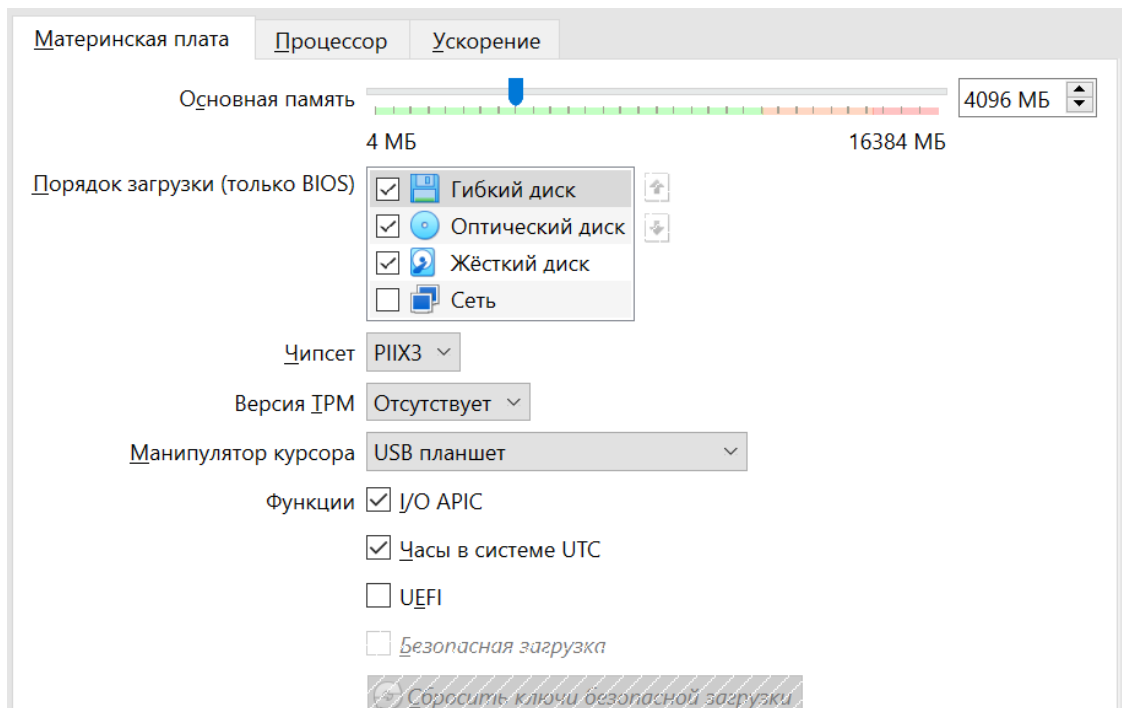


Рисунок 10 — Объем оперативной памяти

Потом создали новый виртуальный жесткий диск (Рисунок 11).

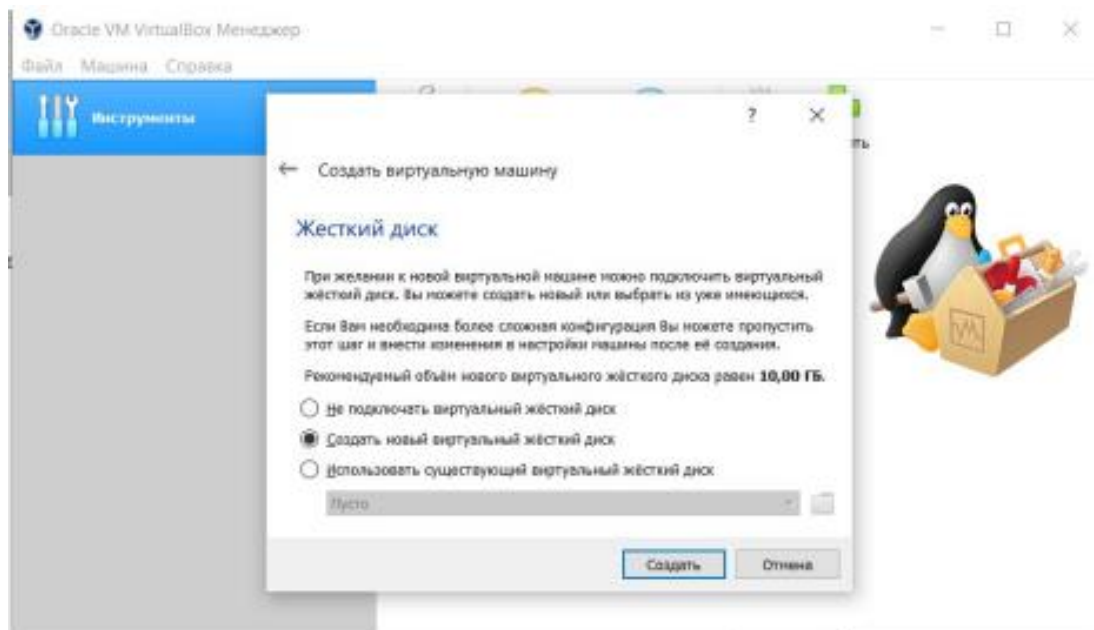


Рисунок 11 — Создание жесткого диска

Затем указали тип нового жесткого диска (Рисунок 12).

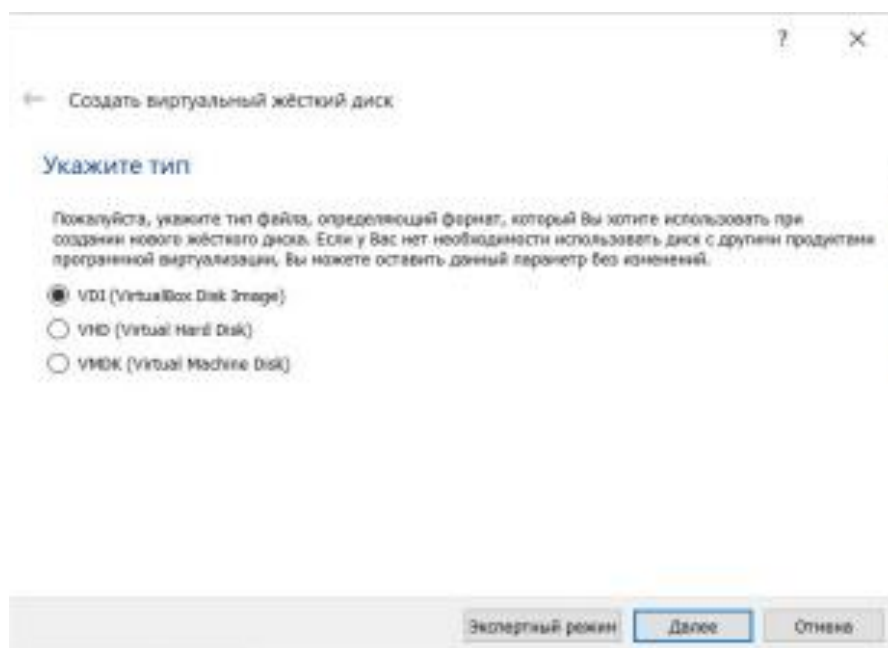


Рисунок 12 — Тип жесткого диска

Далее выбрали формат хранения на виртуальном жестком диске (Рисунок 13).

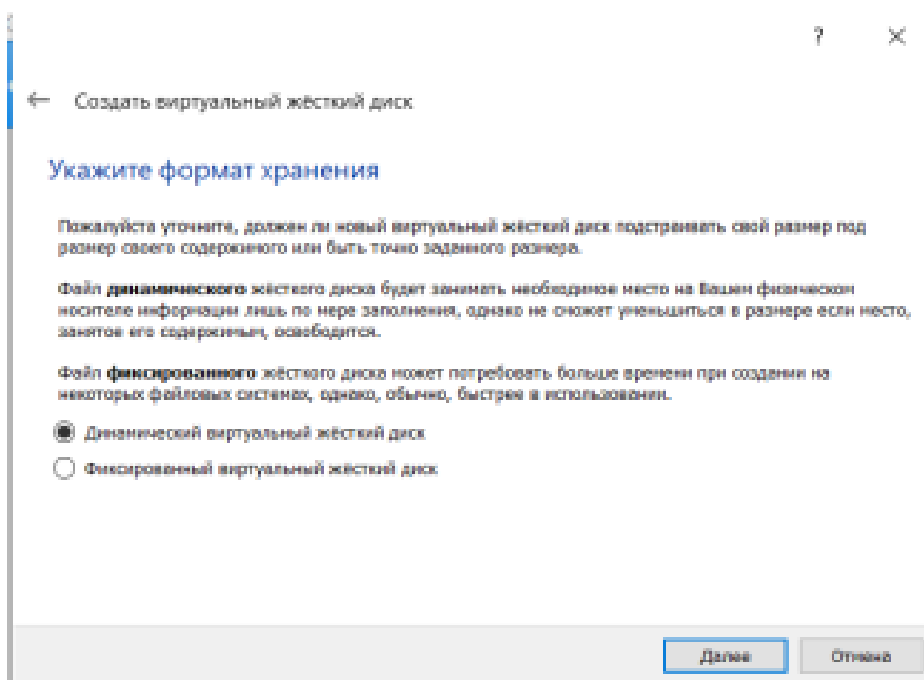


Рисунок 13 — Формат хранения

Потом задали имя нового виртуального жесткого диска и его размер (Рисунок 14).

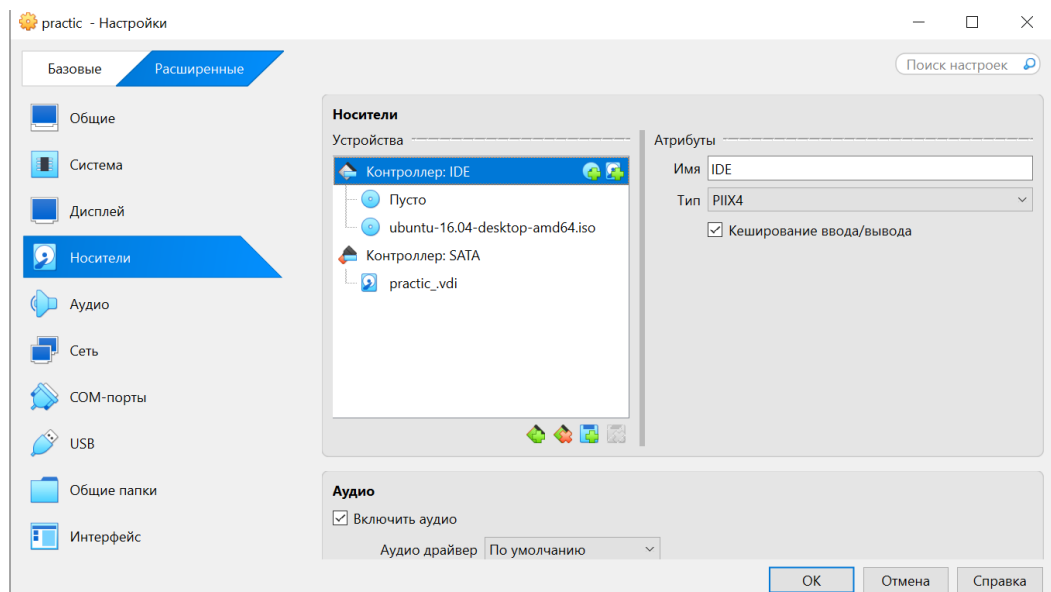


Рисунок 14 — Имя и размер жесткого диск

Виртуальная машина «practic» была создана (Рисунок 15).

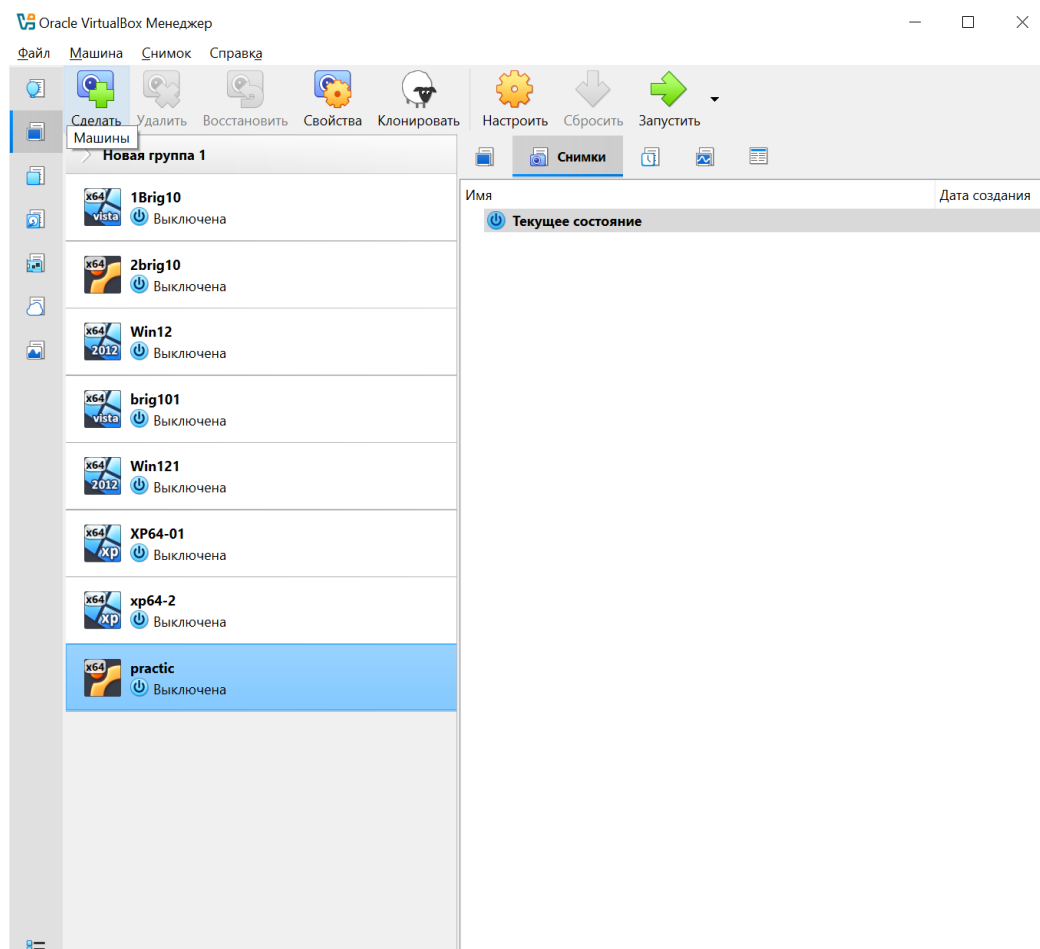


Рисунок 15 — Виртуальная машина «practic»

1.3 Установка ОС Ubuntu на виртуальную машину

Для установки ОС Ubuntu скачали с официального сайта файл, содержащий дистрибутив операционной системы, и запустили созданную виртуальную машину «practic». Запустили виртуальную машину и выбрали скачанный файл образа диска с ОС Ubuntu. Открылся установщик операционной системы (Рисунок 16).

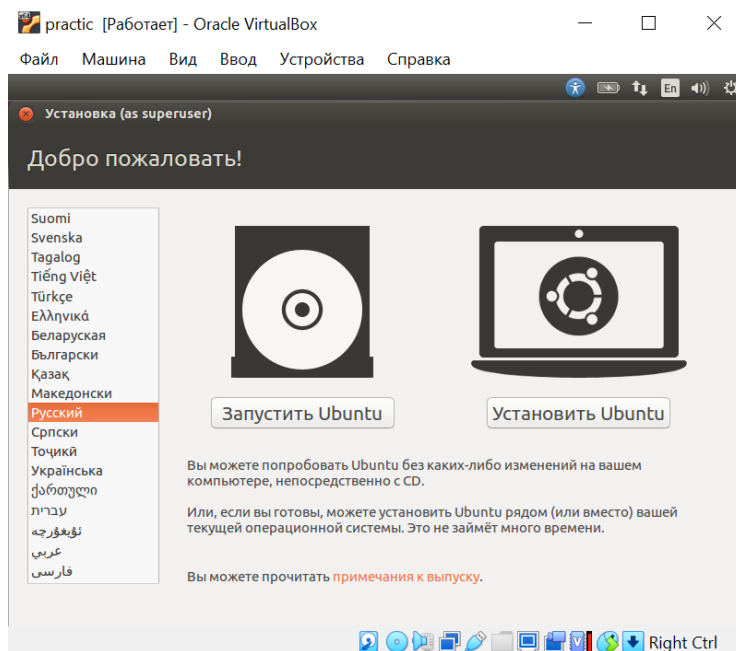


Рисунок 16 — Запуск установщика Ubuntu

Нажали «Установить Ubuntu» и в открывшейся вкладке выбрали пункт «Загрузить обновления во время установки Ubuntu» (Рисунок 17).

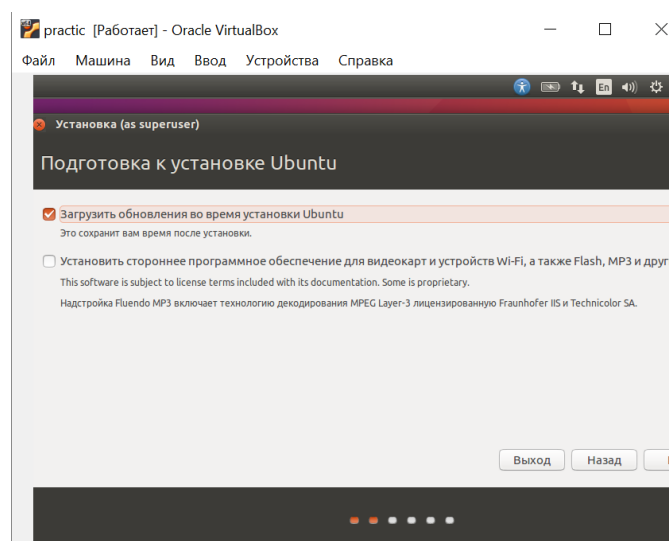


Рисунок 17 — Подготовка к установке Ubuntu

Во вкладке «Тип установки» выбрали «Стереть диск и установить Ubuntu» (Рисунок 18).

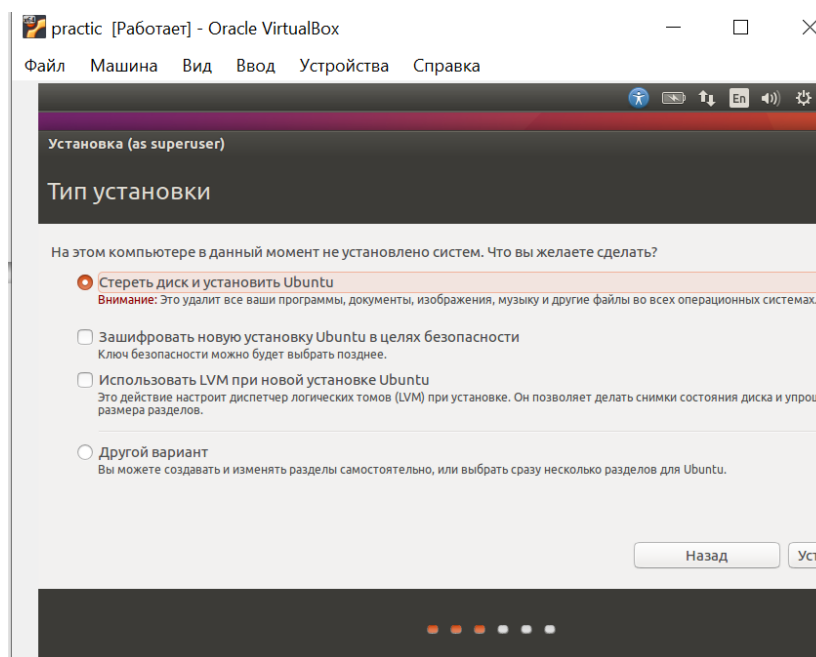


Рисунок 18 — Тип установки Ubuntu

Затем началась установка Ubuntu (Рисунок 19).

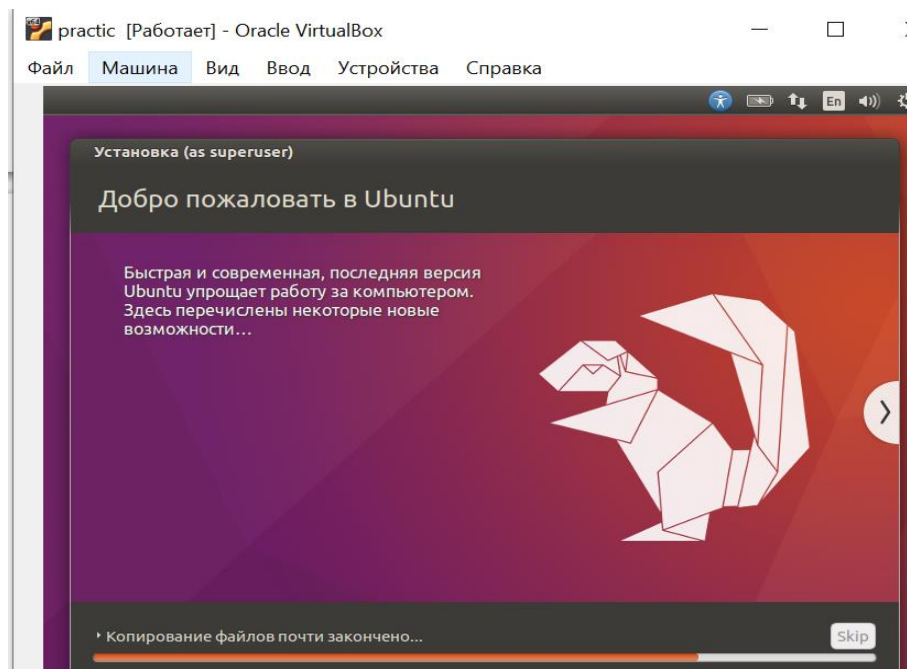


Рисунок 19 — Установка Ubuntu

Установка операционной системы Ubuntu завершена, выбрали перезагрузку виртуальной машины (Рисунок 20).

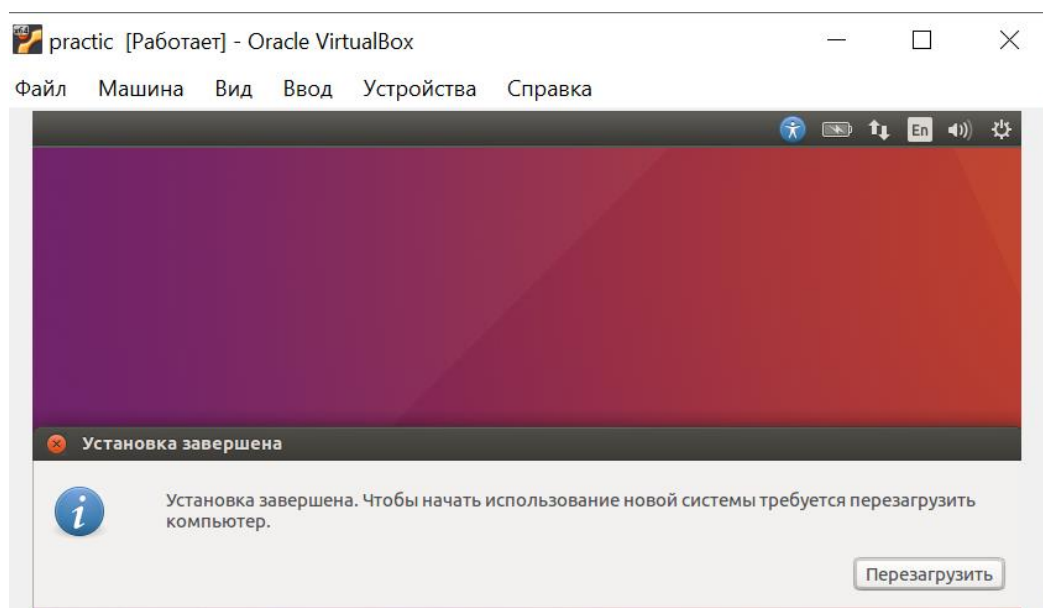


Рисунок 20 — Перезагрузка виртуальной машины

Затем запустилась Ubuntu, ввели пароль для входа в систему (Рисунок 21).

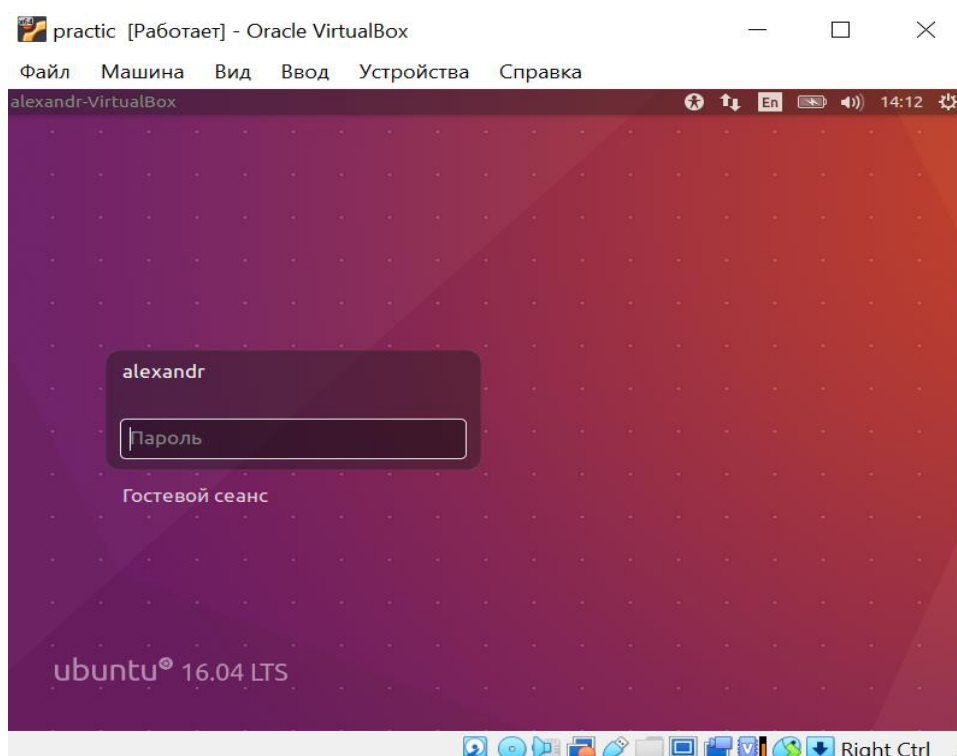


Рисунок 21 — Ввод пароля

ОС Ubuntu готова к работе (Рисунок 22).

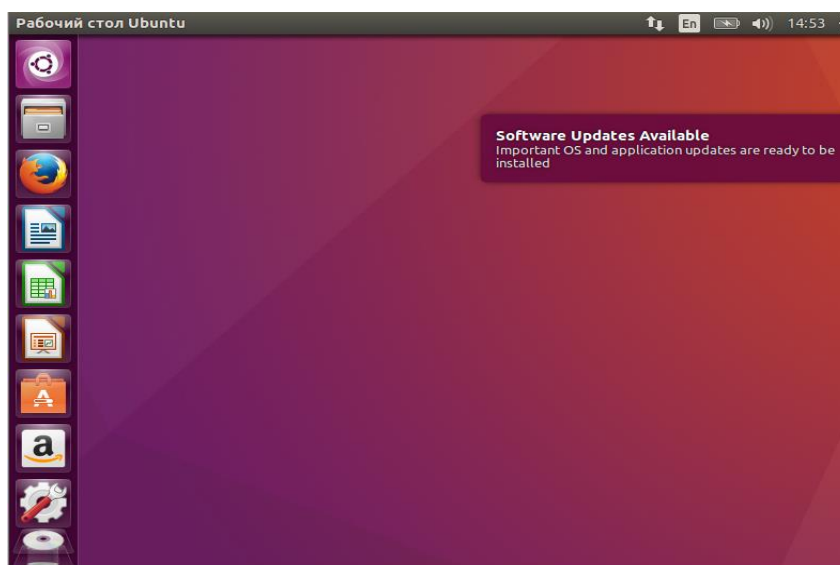


Рисунок 22 — ОС Ubuntu

1.4 ОС Ubuntu

Ubuntu представляет собой операционную систему, построенную на основе Debian GNU/Linux, и является одним из наиболее востребованных дистрибутивов Linux в мире. Данная ОС разработана с акцентом на максимальное удобство и простоту использования для конечного пользователя, что делает её отличным выбором как для начинающих, так и для опытных специалистов. Важной особенностью Ubuntu является широкое применение утилиты `sudo`, предоставляющей пользователям возможность выполнять административные задачи без необходимости входа в систему под учетной записью суперпользователя. Кроме того, дистрибутив обладает развитой системой интернационализации, поддерживая множество языков и региональных настроек.

К числу ключевых преимуществ дистрибутива Ubuntu можно отнести следующие характеристики. Прежде всего, система отличается высокой стабильностью работы и не требует частых перезагрузок, что особенно важно для серверных решений и длительных сессий работы. Безопасность является еще одним значимым достоинством — операционная система спроектирована таким образом, что необходимость в установке антивирусного программного

обеспечения практически отсутствует благодаря продуманной модели прав доступа и регулярным обновлениям безопасности. Важным фактором является и то, что Ubuntu распространяется совершенно бесплатно, что делает её доступной для широкого круга пользователей.

Интерфейс операционной системы отличается приятным визуальным оформлением и интуитивной понятностью, при этом пользователям доступно большое количество бесплатных тем оформления и визуальных эффектов, позволяющих индивидуализировать рабочее окружение. Процесс установки Ubuntu выполняется быстро и просто, что делает систему доступной даже для неопытных пользователей, причем основное программное обеспечение и драйверы устанавливаются автоматически, а система обновлений настроена по умолчанию. Для управления программным обеспечением предусмотрен специализированный менеджер пакетов, через который осуществляется установка большинства приложений. Операционная система предъявляет умеренные требования к аппаратным ресурсам компьютера, что позволяет использовать её на различных конфигурациях оборудования. Наконец, модульная архитектура Ubuntu дает пользователю возможность гибко изменять структуру и компоненты системы вплоть до модификации самого ядра.

2. Интегрированная среда разработки Qt Creator

2.1 Описание Qt Creator

Qt Creator представляет собой кроссплатформенную свободно распространяемую интегрированную среду разработки, предназначенную для создания приложений на языках программирования C, C++ и QML. Основное назначение данного инструмента заключается в максимальном упрощении процесса разработки приложений с использованием фреймворка Qt на различных операционных платформах, включая Windows, Linux и Mac OS. Среда разработки тесно интегрирована с кроссплатформенными системами автоматизации сборки, среди которых основными являются qmake и CMake, что обеспечивает гибкость и удобство настройки процесса компиляции проектов.

Редактор кода, входящий в состав Qt Creator, оснащен широким набором функциональных возможностей, повышающих эффективность работы программиста. В их число входит интеллектуальное автодополнение, распространяющееся в том числе на ключевые слова языка, подсветка синтаксиса, наглядное отображение сообщений об ошибках и предупреждений непосредственно в процессе написания кода. Кроме того, редактор предоставляет возможность тонкой настройки стиля выравнивания текста, управления отступами и автоматической расстановки скобок, что способствует поддержанию единообразного и читаемого стиля кодирования. В состав среды встроен Qt Designer, используемый для визуального проектирования и сборки графических интерфейсов пользователя из стандартных виджетов Qt, что значительно ускоряет разработку приложений с графическим интерфейсом.

Qt Creator обладает развитой интеграцией с набором полезных инструментов, расширяющих возможности разработчика. В их число входят системы управления версиями, позволяющие эффективно отслеживать изменения в исходном коде и работать в команде, а также эмулятор Qt, дающий возможность тестировать приложения для мобильных платформ без использования физических устройств. Важной функцией среды является поддержка отладки приложений, для чего предоставляются интерфейсы к GNU

Symbolic Debugger (GDB) и Microsoft Console Debugger (CDB) для отладки обычных приложений на C++, а также внутренние отладчики для JavaScript, что обеспечивает комплексный подход к поиску и исправлению ошибок.

Среда разработки предоставляет полноценную поддержку для сборки и запуска приложений на Qt как для настольных компьютеров, работающих под управлением Windows, Linux и Mac OS, так и для мобильных устройств на базе Symbian, Maemo и MeeGo. Гибкие настройки сборки позволяют разработчику быстро переключаться между различными целевыми платформами и конфигурациями, что существенно упрощает процесс создания кроссплатформенных приложений и тестирования их в различных окружениях. Благодаря интеграции всех необходимых инструментов в единую среду, Qt Creator становится удобным и эффективным решением для профессиональной разработки программного обеспечения на языке C в операционной системе Linux.

2.2 Описание установки

Для установки интегрированной среды разработки Qt Creator в операционной системе Ubuntu открыли Терминал и написали в нем необходимые команды (Рисунок 23).

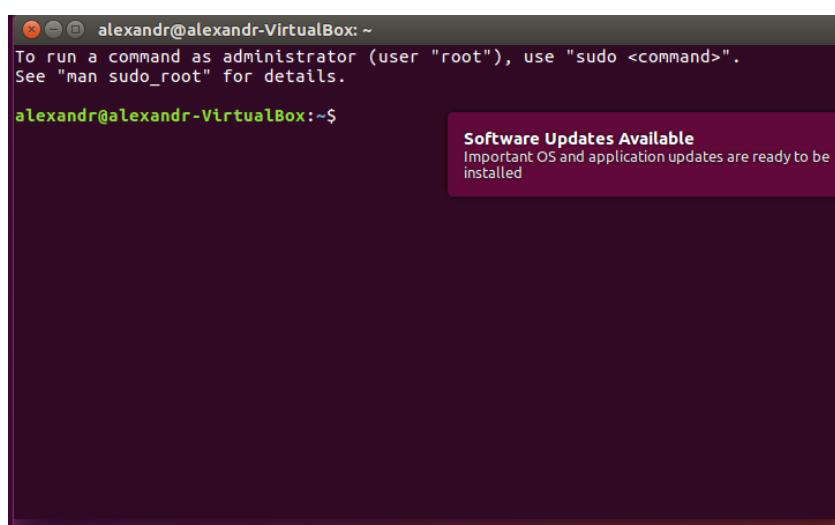


Рисунок 23 — Открытие терминала

Команду `sudo apt update` использовали для установки обновлений списка пакетов. Команду `sudo apt install qt5-default` использовали для установки Qt. Команду `sudo apt install qtcreator` использовали для установки Qt Creator. Команду `sudo apt install qtbase5-examples qtdeclarative5-examples` использовали для установки примеров программ. Установка Qt Creator была завершена, далее открыли среду (Рисунок 24).

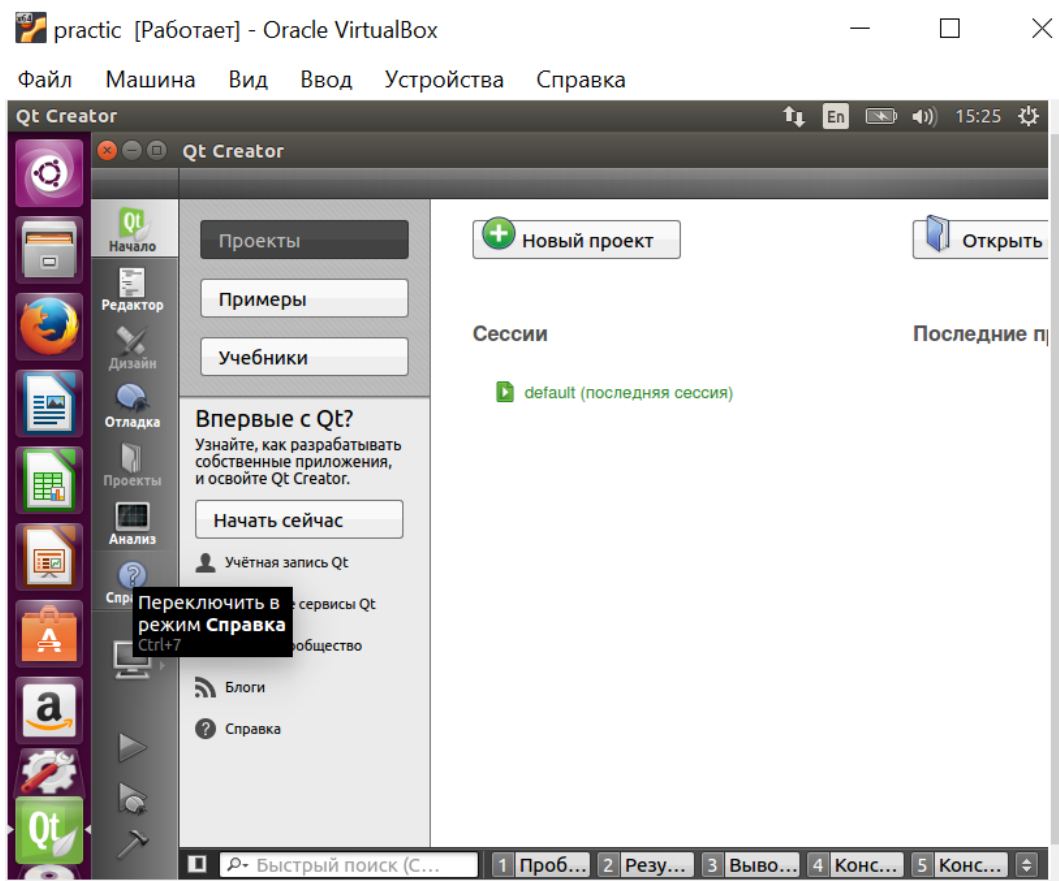


Рисунок 24 — Открытие Qt Creator

3. Разработка программы на языке C в среде Qt Creator

3.1 Формулировка задачи

Разработать программу, которая запрашивает у пользователя количество элементов массива и сами элементы массива, вводимые с клавиатуры, после чего выполняет сортировку данного массива по возрастанию методом пузырька (bubble sort). Программа должна выводить на экран исходный и отсортированный массивы, а также сохранять результат сортировки в текстовый файл. В случае возникновения ошибки при создании файла программа должна выводить соответствующее сообщение.

3.2 Создание проекта в Qt Creator

Для создания проекта в Qt Creator запустили среду разработки и на начальной странице нажали кнопку «Создать». Затем выбрали «Приложение на языке C» (Рисунок 25).

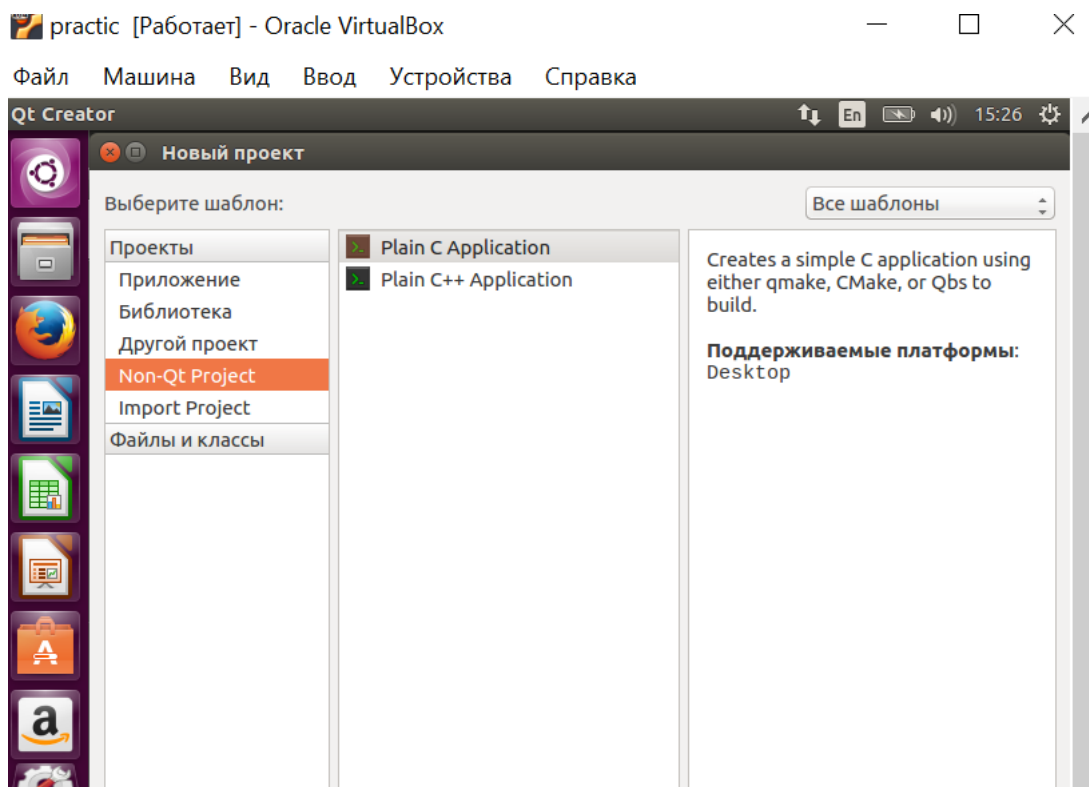


Рисунок 25 — Выбор создаваемого приложения

Далее указали название и размещение проекта (Рисунок 26).

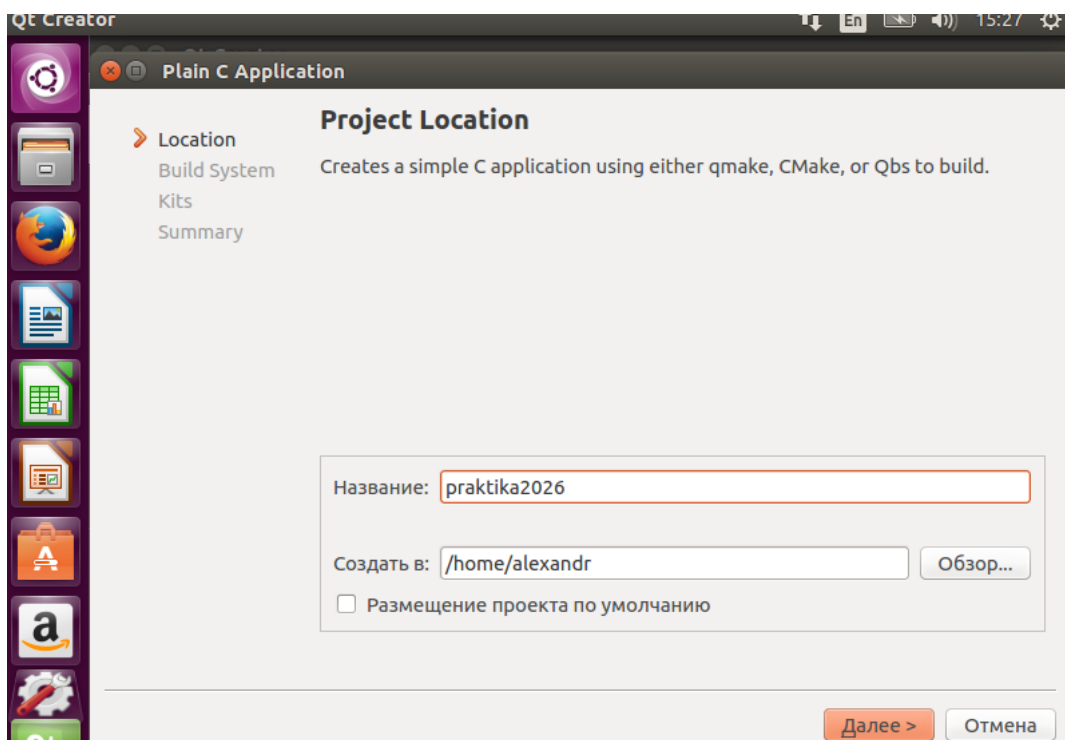


Рисунок 26 — Имя и размещение проекта

Потом выбрали систему сборки проекта (Рисунок 27).

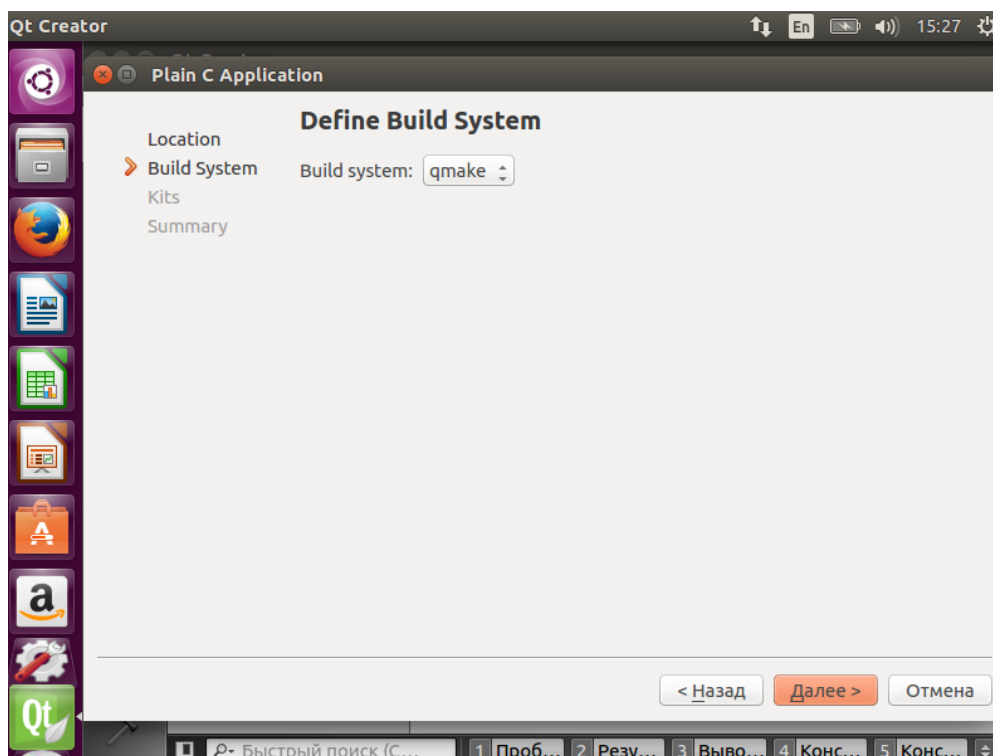


Рисунок 27 — Выбор системы сборки

Затем выбрали комплекты проекта (Рисунок 28).

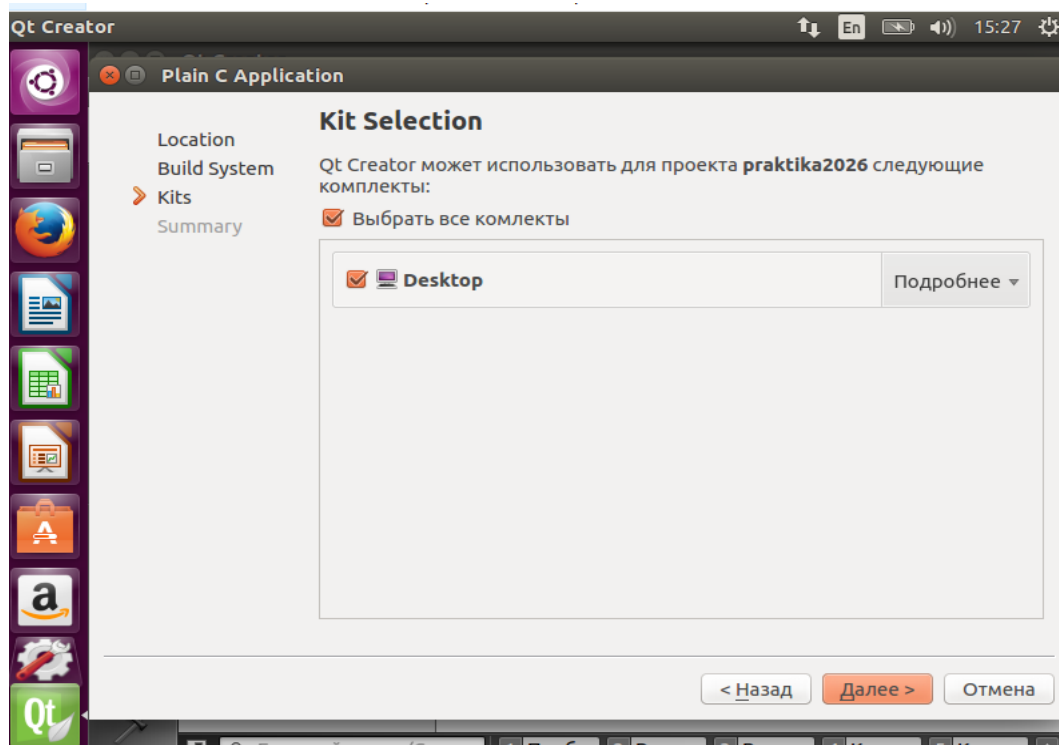


Рисунок 28 — Выбор комплекта

Далее указали управление проектом и нажали кнопку «Завершить» (Рисунок 29).

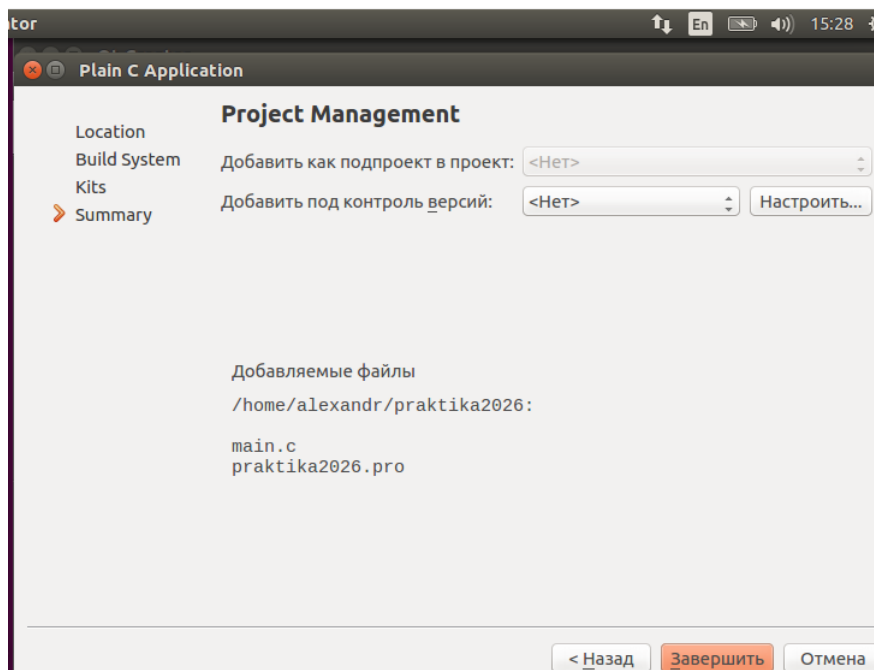


Рисунок 29 — Управление проектом

Проект создан, открылся редактор для написания кода программы (Рисунок 30).

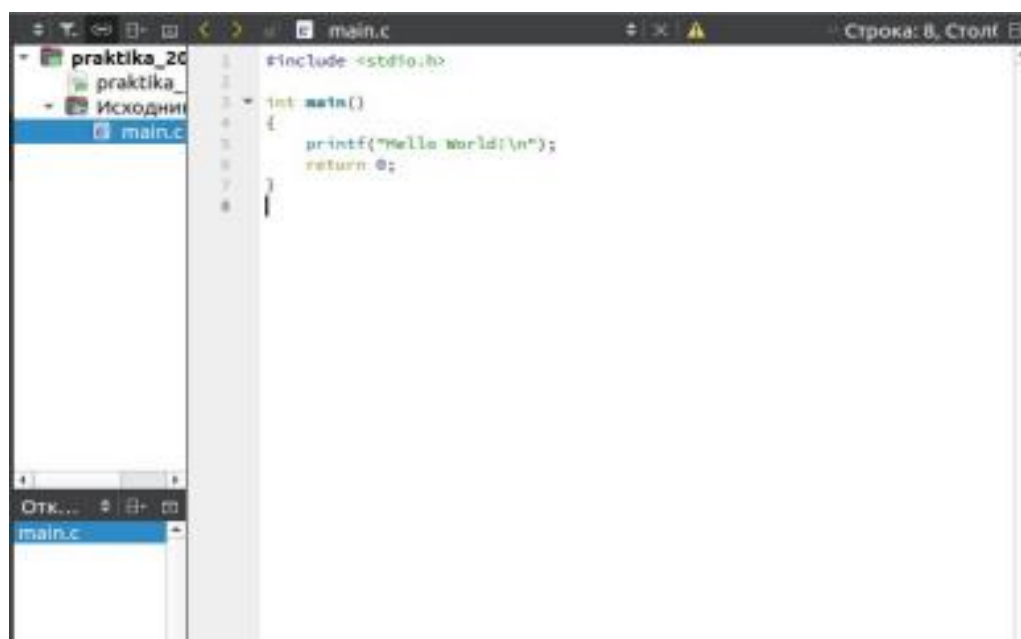


Рисунок 30 — Открытие редактора

3.3 Описание программы

Решение задачи начали с подключения заголовочного файла `stdio.h`, который необходим для использования функций ввода-вывода. Далее объявили функцию `bubbleSort`, которая принимала массив целых чисел и его размер. Внутри этой функции объявили переменные `i`, `j` и `temp`. С помощью двух вложенных циклов `for` организовали сортировку методом пузырька: внешний цикл проходил по всем элементам массива, а внутренний цикл попарно сравнивал соседние элементы и менял их местами, если левый элемент был больше правого. Временная переменная `temp` использовалась для хранения значения одного из элементов при обмене.

После этого объявили функцию `printArray`, которая принимала массив и его размер и выводила все элементы массива на экран через пробел. Внутри функции использовали цикл `for` для перебора всех элементов и функцию `printf` для вывода каждого элемента.

В функции `main` начиналась непосредственная работа программы. Сначала объявили переменные `n` и `i` целочисленного типа. Затем с помощью функции

printf вывели сообщение пользователю с просьбой ввести количество элементов массива, и с помощью функции scanf это значение считали и сохранили в переменную n. Далее объявили массив arr размером n — это динамический массив, размер которого определялся во время выполнения программы.

Затем вывели сообщение с просьбой ввести элементы массива, и с помощью цикла for и функции scanf произошло последовательное считывание всех элементов массива и сохранение их в ячейки arr[i]. После ввода всех элементов с помощью функции printArray вывели исходный массив на экран. Затем вызвали функцию bubbleSort, которая отсортировала массив по возрастанию. После сортировки снова вызвали функцию printArray и вывели отсортированный массив на экран.

Далее объявили указатель file типа FILE. С помощью функции fopen открыли файл с именем "bubble_result.txt" в режиме записи. Проверили, удалось ли открыть файл: если указатель не был равен NULL, то с помощью функции fprintf в файл записали строку с текстом и все элементы отсортированного массива через пробел. После записи файл закрыли с помощью функции fclose, и на экран вывели сообщение о том, что результат успешно записан в файл. Если же файл открыть не удалось, выводилось сообщение об ошибке. В конце программа возвращала 0, что означало успешное завершение работы.

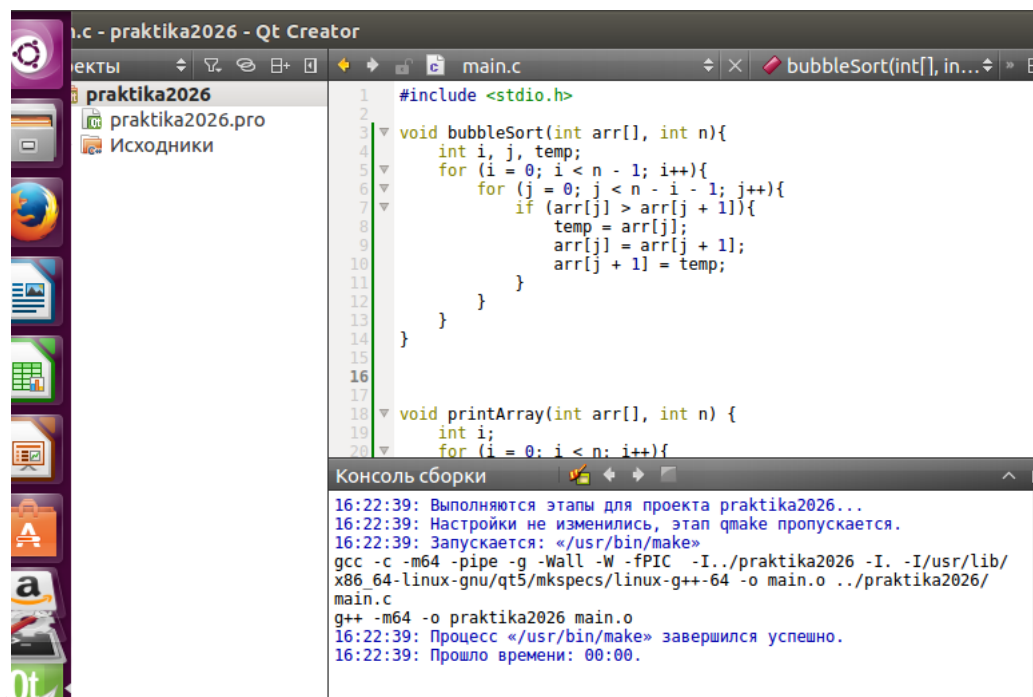
3.4 Компиляция и отладка программы в среде Qt Creator

Для компиляции программ в Qt Creator использовали встроенный компилятор для языка C — Clang, который находится в системной директории /usr/bin. Данный компилятор позволял транслировать исходный код программы на языке C в машинный код, понятный процессору компьютера. В среде Qt Creator для сборки проектов применялись системы автоматизации сборки CMake и QMake, которые управляли процессом компиляции и связывания файлов проекта.

Для того, чтобы собрать разработанную программу, в главном меню среды разработки нажимали кнопку «Собрать проект» или пользовались комбинацией

клавиш **Ctrl + B**. После запуска процесса сборки компилятор обрабатывал все файлы исходного кода, проверял их на наличие синтаксических и логических ошибок и создавал исполняемый файл программы. В процессе сборки все сообщения, включая информацию о ходе компиляции, предупреждения и ошибки, выводились в специальное окно «Вывод сборки».

Если компиляция проходила успешно, в этом окне отображалось соответствующее сообщение об успешном завершении сборки и создании исполняемого файла. В случае возникновения ошибок в коде, компилятор указывал на конкретные строки и типы ошибок, что помогало быстро их исправить. После успешной сборки программу можно было запустить на выполнение непосредственно из среды разработки и протестировать её работу (Рисунок 31).



The screenshot shows the Qt Creator IDE interface. The left sidebar displays the project structure for 'praktika2026', including 'praktika2026.pro' and 'Исходники'. The main editor window shows the source code for 'main.c', which includes a bubble sort algorithm and a printArray function. The bottom panel, titled 'Консоль сборки' (Build Console), displays the compilation output:

```
16:22:39: Выполняются этапы для проекта praktika2026...
16:22:39: Настройки не изменились, этап qmake пропускается.
16:22:39: Запускается: «/usr/bin/make»
gcc -c -m64 -pipe -g -Wall -W -fPIC -I../praktika2026 -I. -I/usr/lib/
x86_64-linux-gnu/qt5/mkspecs/linux-g++-64 -o main.o ../praktika2026/
main.c
g++ -m64 -o praktika2026 main.o
16:22:39: Процесс «/usr/bin/make» завершился успешно.
16:22:39: Прошло времени: 00:00.
```

Рисунок 31 — Вывод результата сборки проекта

При запуске проекта производилась его компиляция. Для запуска программы нажимали кнопку «Запустить» или комбинацию клавиш Ctrl + R (Рисунок 32).

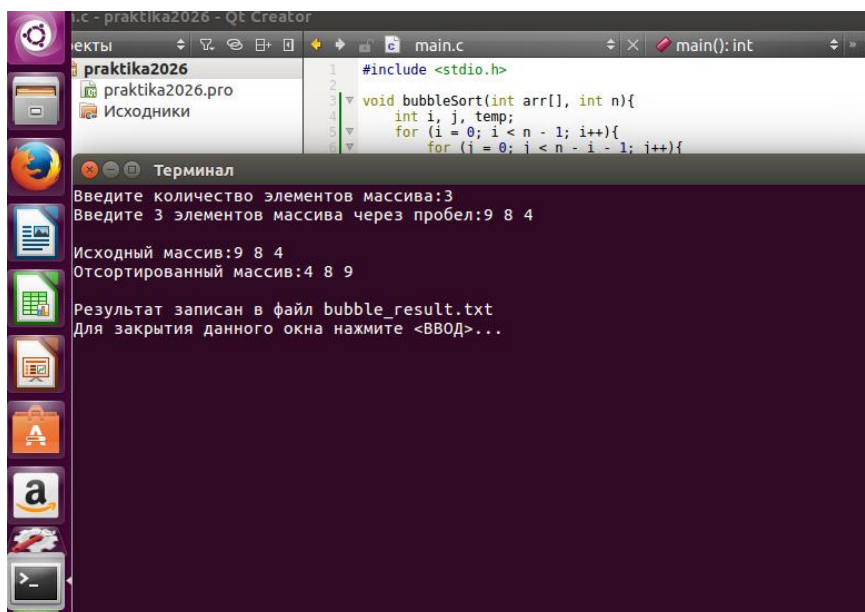


Рисунок 32 — Вывод результата сборки проекта

Для отладки проекта в среде Qt Creator использовали встроенный отладчик GDB, расположенный в /usr/bin. Для запуска отладчика нажимали кнопку «Начать отладку запускающего проекта». Также ставили точку останова на любую строку кода. Для завершения отладки нажимали кнопку «Остановить отладчик» и убирали точку останова, если она была поставлена (Рисунок 33).

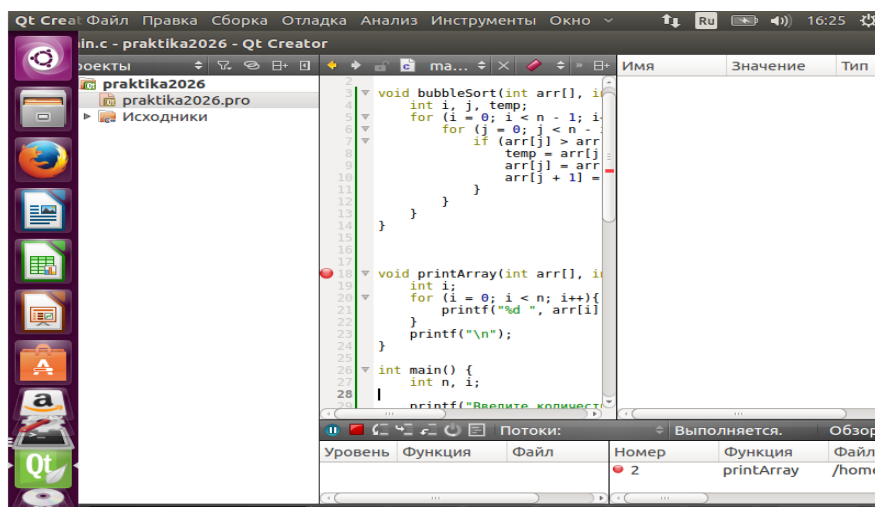


Рисунок 33 — Запуск отладчика и установка точки останова

Заключение

В ходе выполнения практической работы была изучена и установлена операционная система Ubuntu на виртуальную машину VirtualBox, установлена интегрированная среда разработки Qt Creator в ОС Ubuntu, были изучены основные инструменты среды и разработана программа на языке C в среде Qt Creator. Результаты испытаний показали, что программа работает согласно формулировке задачи. В процессе работы были получены практические навыки установки и настройки операционной системы Linux, освоены базовые принципы работы с виртуальными машинами, изучены основные компоненты и структура операционной системы Ubuntu. Также были приобретены навыки работы в интегрированной среде разработки Qt Creator, включая создание проекта, написание кода, компиляцию, отладку и запуск программ на языке программирования C. Разработанная программа успешно выполняет сортировку массива методом пузырька, корректно выводит исходные и отсортированные данные на экран, а также сохраняет результаты работы в файл, что подтверждает правильность реализации алгоритма и функционирования программы в целом. Таким образом, все поставленные задачи практической работы были выполнены в полном объеме, а полученные знания и умения могут быть применены в дальнейшей профессиональной деятельности в области разработки программного обеспечения и системного администрирования.

Список использованных источников и литературы

1. Водолазкий, В.В. Путь к Linux. Учебный курс / В.В. Водолазкий. - М.: СПб: Питер; Издание 3-е, 2021. - 400
2. Таненбаум, Э. Современные операционные системы / Э. Таненбаум, Х. Бос. – 4-е изд. – СПб. : Питер, 2015. – 1120 с.
3. Колисниченко, Д. Н. Ubuntu 10. Быстрый старт / Д. Н. Колисниченко. – СПб. : БХВ-Петербург, 2010. – 272 с.
4. Браун, М. Введение в Linux / М. Браун. – М. : ЭКОМ, 2009. – 312 с.
5. Керниган, Б. У. Язык программирования С / Б. У. Керниган, Д. М. Ритчи. – 2-е изд. – М. : Вильямс, 2013. – 304 с.

Приложение А

Листинг программы

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void printArray(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int n, i;

    printf("Введите количество элементов массива: ");
    scanf("%d", &n);

    int arr[n];

    printf("Введите %d элементов массива через пробел: ", n);
```

```

    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("\nИсходный массив: ");
    printArray(arr, n);

    bubbleSort(arr, n);

    printf("Отсортированный массив: ");
    printArray(arr, n);

    FILE *file;
    file = fopen("bubble_result.txt", "w");

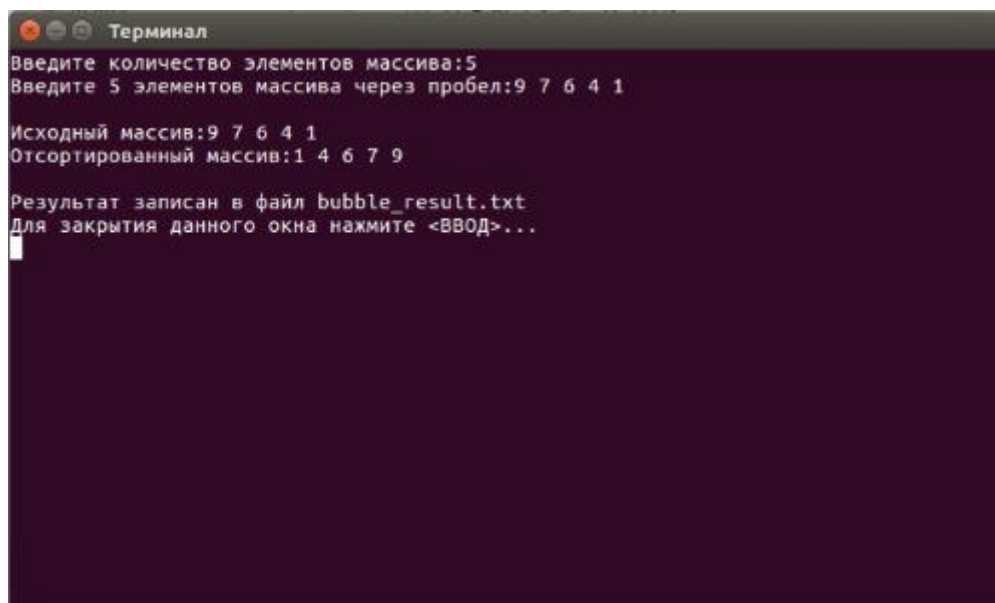
    if (file != NULL) {
        fprintf(file, "Отсортированный массив: ");
        for (i = 0; i < n; i++) {
            fprintf(file, "%d ", arr[i]);
        }
        fprintf(file, "\n");
        fclose(file);
        printf("\nРезультат записан в файл bubble_result.txt\n");
    } else {
        printf("Ошибка: не удалось создать файл!\n");
    }

    return 0;
}

```

Приложение Б

Результаты тестирования



```
Терминал
Введите количество элементов массива:5
Введите 5 элементов массива через пробел:9 7 6 4 1

Исходный массив:9 7 6 4 1
Отсортированный массив:1 4 6 7 9

Результат записан в файл bubble_result.txt
Для закрытия данного окна нажмите <ВВОД>...
```

Рисунок Б.1 — Ввод: 9 7 6 4 1



```
Терминал
Введите количество элементов массива:3
Введите 3 элементов массива через пробел: 2 3 1

Исходный массив:2 3 1
Отсортированный массив:1 2 3

Результат записан в файл bubble_result.txt
Для закрытия данного окна нажмите <ВВОД>...
```

Рисунок Б.2 — Ввод: 2 3 1



Рисунок Б.3 — Проверка записи результата в файл